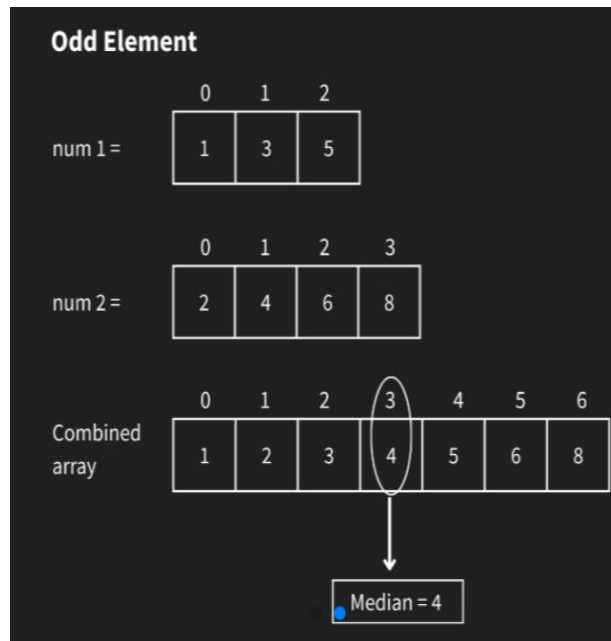
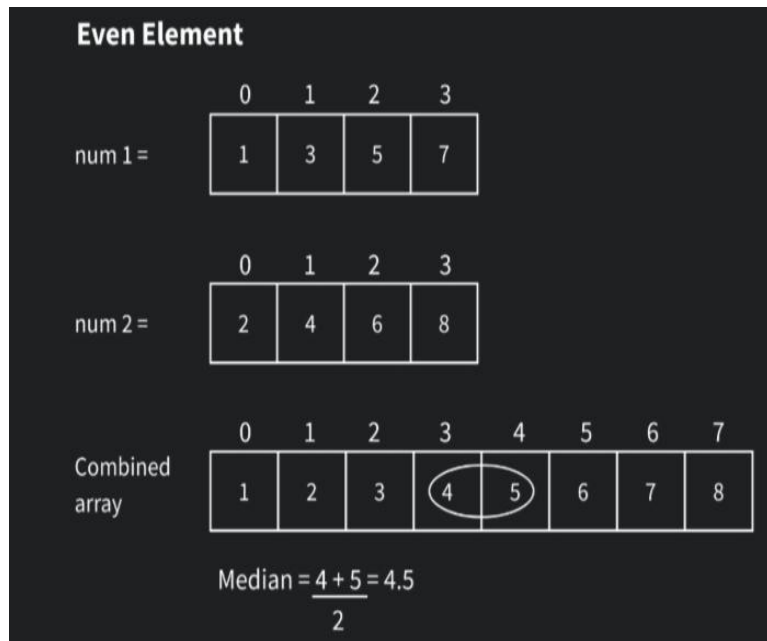


Binary Search & Parametric Search

1. Median of 2 sorted Arrays

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays. The overall run time complexity should be $O(\log(m+n))$.



Example 1:

Input: nums1 = [1,3], nums2 = [2]

Output: 2.00000

Explanation: merged array = [1,2,3] and median is 2.

Example 2:

Input: nums1 = [1,2], nums2 = [3,4]

Output: 2.50000

Explanation: merged array = [1,2,3,4] and median is $(2 + 3) / 2 = 2.5$.

Approach:

Binary Search on the Smaller Array :

We split the combined arrays into two halves — left and right.

If the total number of elements is odd, the left half will have one extra element.

All values in the left half must be less than or equal to those in the right half.

If total elements are odd, the median is the last element of the left half.

If they are even, the median is the average of the last element from the left and the first from the right.

1. Start with the smaller array - To make things faster and easier to handle, we always apply binary search on the array with fewer elements. If the first array is longer, we simply swap them by calling the same function with the two arrays flipped.

2. Set the binary search limits : We begin binary search on the shorter array using two pointers: low = 0 and high = size of first array.

3. Divide both arrays logically

In each binary search step:

Pick a position i_1 in the first array.

Then figure out a matching position i_2 in the second array so that the left half across both arrays contains half the total elements.

4. Get values around the cut : Based on the positions $i1$ and $i2$, we look at the elements just before and after the cut in both arrays. If the cut goes past the edge of an array, we treat it as having $-\infty$ on the left or $+\infty$ on the right to avoid out-of-bounds errors.

5. Check if the split is valid

If the largest value on the left of both cuts is less than or equal to the smallest value on the right, we've found the correct split:

If the total number of elements is even, the median is the average of the two middle values.

If it's odd, the median is simply the maximum of the left half.

6. Adjust the search window if needed

If the value from the first array's right side is too small, we shift the search to the right ($low = i1 + 1$).

Otherwise, we move it left ($high = i1 - 1$) to bring the partition into balance.

2. Partition Problem(Painter's Partition)

Given an array `arr[]` and `K`, where the array represents the boards and each element of the given array represents the length of each board. `K` numbers of painters are available to paint these boards. Consider that each unit of a board takes 1 unit of time to paint. The task is to find the minimum time to get this job done by painting all the boards under the constraint that any painter will only paint the continuous sections of boards.

Minimum Time to Paint All Boards:

Rohan needs to get several wooden panels painted, each with its own length. He hires `K` painters to finish the task quickly, where each unit of length takes 1 unit of time. There's one condition: every painter can only work on panels that are next to each other. Since everyone starts at the same time, the total time depends on whoever finishes last. Rohan wants to split the work in the most efficient way to finish the job early. So, the challenge is to divide the boards to minimize the time taken by the busiest painter. This is the exact problem we're solving.

Possible Allocations:



Minimizing the maximum time

Example 1:

Input: `Arr[] = [10, 30, 20, 40]`, `k = 2`

Output: 60

Explanation:

The most optimal way will be:

Painter 1 allocation: [10, 30, 20]

Painter 2 allocation: [40]

Job will be done when all painters finish, at time
 $= \max(10 + 30 + 20, 40) = \max(60, 40) = 60$

Approach:

Optimized Time Allocation using Binary Search :

If there is only 1 painter (i.e., $k = 1$), then that painter must paint all the boards, so the total time will be the sum of the array.

If there are n painters (i.e., $k = n$), then each painter can take one board, and the time taken will be the maximum board length.

So, the valid time range lies between `maxValue` and `totalSum`.

Now, we apply binary search in this range to find the minimum time required to paint all boards using k painters

1. Establish the parameters of the search:

Go through the array to find:

`maxVal`: the largest board (at least this amount must be painted by a painter).

`totalSum`: the sum of all the boards (the longest time if a single painter handles it all).

2. Apply binary search within `maxVal` and `totalSum`

To determine the least amount of time needed, experiment with different values between `maxVal` and `totalSum`.

3. Verify the correctness of each mid-value:

Use `isValid()` to check if we can split the task among k painters without going over `mid` for each estimated time (`mid`).

4. Adjust the range based on the result:

If mid works, store it and try a lower value ($\text{high} = \text{mid} - 1$) .

If its fails, Increase the range ($\text{low} = \text{mid} + 1$) to give yourself more time .

5. Return the best time found:

The result will contain the least amount of time that satisfies all requirements after the loop is finished.

3. Aggressive Cows Problem

Given an array stalls[] containing distinct integers representing stall locations and an integer k representing the number of aggressive cows, your task is to assign stalls to the cows such that the minimum distance between any two cows is maximized.

Example 1:

Input: stalls[] = [6, 1, 9, 3, 8], k = 3

Output: 3

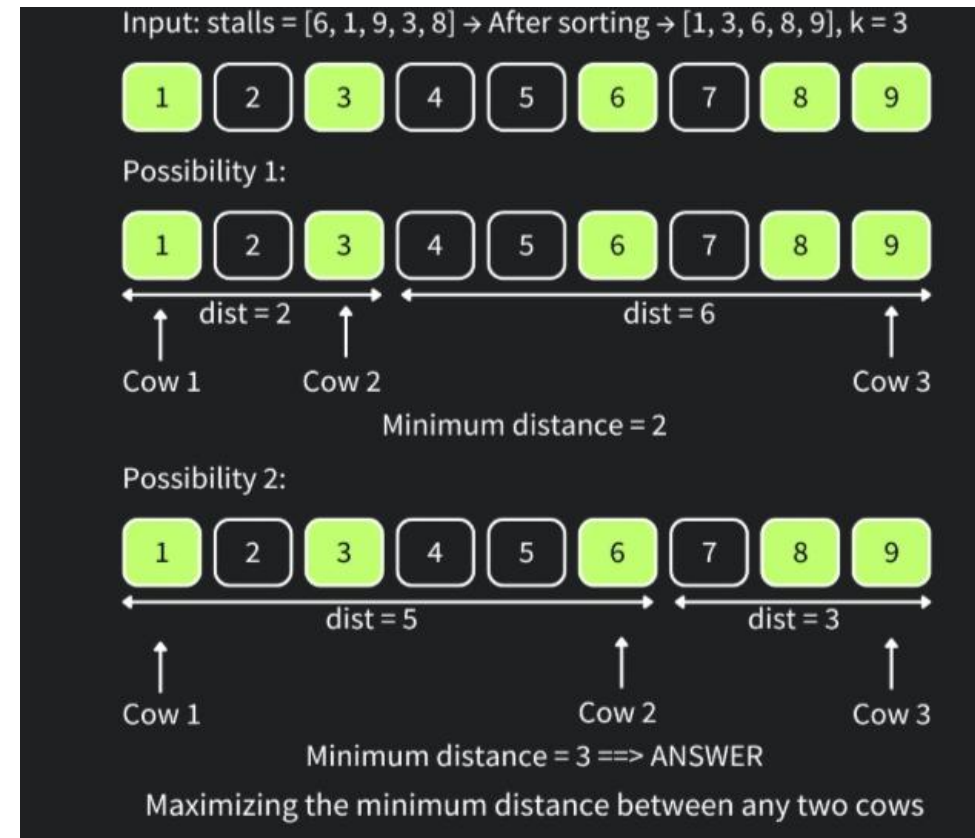
Explanation:

After sorting the stalls, we get: [1, 3, 6, 8, 9].

We can place the first cow at stalls[0] = 1,
the second cow at stalls[2] = 6,
and the third cow at stalls[4] = 9.

The minimum distance between any two cows is $\min(6 - 1, 9 - 6) = 3$.

This is the largest possible minimum distance among all valid placements.



Approach:

Binary Search :

To stop cows from getting too close and fighting, we need to spread them out as much as possible across the stalls. By sorting the stall positions, we can try different gaps and see what works. Using binary search, we test each possible gap and check if all cows can be placed with that spacing. The biggest distance that still allows all cows to fit becomes our final answer.

1. Sort the stall positions

We sort the array so that cows can be arranged in increasing order of stall places for easy distance comparisons.

2. Define the search space for minimum distance.

The smallest possible distance between any two cows is 1, stalls[n - 1] - stalls[0], is the longest distance that can exist between any two cows.

3. Use binary search to find the maximum possible minimum distance

Instead of using the stall positions, we conduct a binary search on the distance value (mid).

We determine if it is feasible to arrange every cow so that the distance between each pair is at least mid.

4. Validate each distance using a helper function

Place the first cow in the first stall to begin.

Traverse the stalls and place the next cow only if it's at least mid units away from the last placed cow.

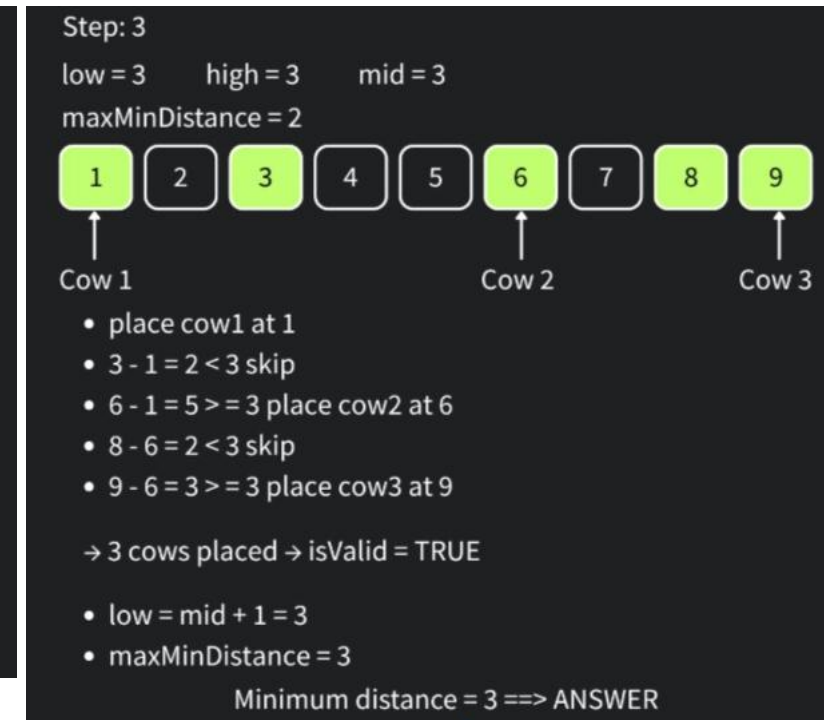
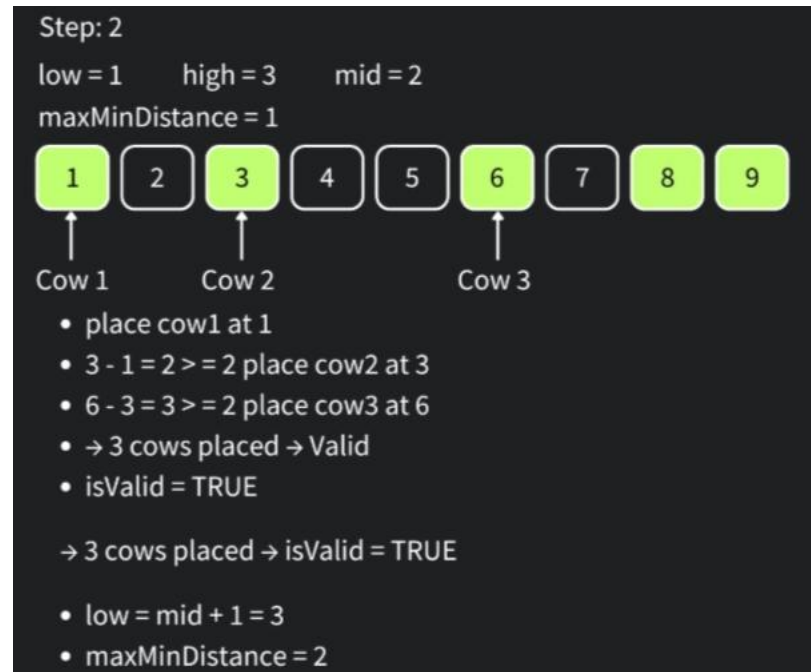
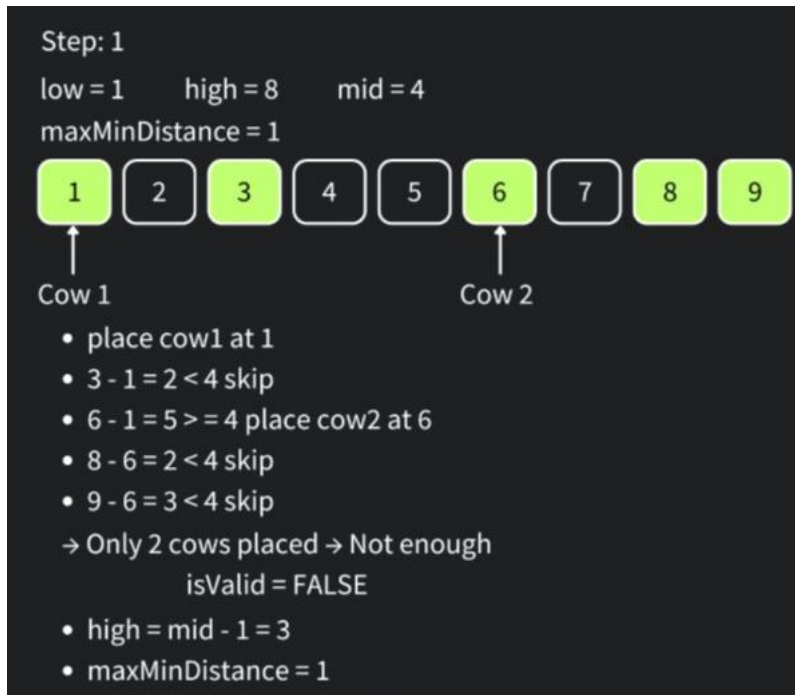
The existing distance is appropriate if we can arrange all of the cows in this manner.

5. Adjust search range based on validation

If it is feasible to place every cow, Store the distance as a possible solution and look for a higher number ($\text{low} = \text{mid} + 1$).
If it isn't possible, reduce the search space ($\text{high} = \text{mid} - 1$).

6. Return the largest valid distance

When $\text{low} > \text{high}$, the loop is terminated, and the result is the most recent valid distance calculated.



4. Book Allocation Problem:

You are given an array `arr[]` of integers, where each element `arr[i]` represents the number of pages in the *i*th book. You also have an integer *k* representing the number of students. The task is to allocate books to each student such that :

Each student receives at least one book.

Each student is assigned a contiguous sequence of books.

No book is assigned to more than one student.

The objective is to minimize the maximum number of pages assigned to any student. In other words, out of all possible allocations, find the arrangement where the student who receives the most pages still has the smallest possible maximum.

Note: Return -1 if a valid assignment is not possible, and the allotment should be in contiguous order (see the explanation for better understanding).

The Fair Book Distribution Problem:

You're a teacher with a stack of books, each having a different number of pages. You have *k* students, and you want to divide the books among them. Each student must get at least one book, and books must be given in order—no skipping around. Also, no two students can share a book. Your goal is to split the books in a way that the student who reads the most pages reads as little as possible. If you have more students than books, it's just not possible—return -1.

The challenge is to find this minimum-maximum page load.

Example 1

Input: arr[] = [12, 34, 67, 90], k = 2

Output: 113

Explanation: Allocation can be done in following ways:

[12] and [34, 67, 90] Maximum Pages = 191

[12, 34] and [67, 90] Maximum Pages = 157

[12, 34, 67] and [90] Maximum Pages = 113.

Therefore, the minimum of these cases is 113, which is selected as the output.

Example 2

Input: arr[] = [15, 17, 20], k = 5

Output: -1

Explanation: Allocation cannot be done.

Example 3

Input: arr[] = [22, 23, 67], k = 1

Output: 112

Explanation : All the books have been given to single student.

Approach:

Binary Search on Minimum Maximum Pages :

We want to split books so that the student with the most pages gets as few as possible.

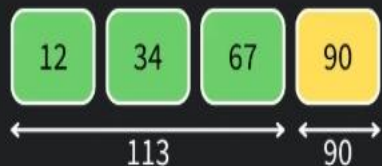
Using binary search, we guess this max limit and check if we can divide books among k students without exceeding it. The smallest such valid guess is our answer.

1. If there are more students than books ($k > n$), it's impossible to assign at least one book to each—return -1.
2. Figure out the limits: The least any student can get is the biggest book. The most is if one student takes all the books.
3. Start a binary search between those two. Try a mid-value as the max pages allowed per student.
4. Go through books. Add pages one by one. If the sum goes over mid, hand over to the next student. Count how many students are needed.
5. If students needed $> k$, the guess is too low—shift right. If it works, save it and search left for a smaller valid answer.
6. When the search ends, return the smallest max found.

Step: 1

low = 90 high = 203 mid = 146

isAllocationPossible() ==> true



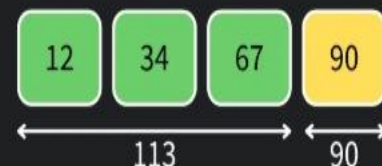
answer = 146

Move high to mid -1 i.e. 145

Step: 2

low = 90 high = 145 mid = 117

isAllocationPossible() ==> true



answer = 117

Move high to mid -1 i.e. 116

Step: 3

low = 90 high = 116 mid = 103

isAllocationPossible() ==> false



✗ Allocation not possible with 103 pages – last book remains unassigned.

Move low to mid +1 i.e. 104

Step: 4

low = 104 high = 116 mid = 110

isAllocationPossible() ==> false



✗ Allocation not possible with 110 pages – last book remains unassigned.

Move low to mid +1 i.e. 111

Step: 5

low = 111 high = 116 mid = 113

isAllocationPossible() ==> true



answer = 113

Move high to mid -1 i.e. 112

Step: 6

low = 111 high = 112 mid = 111

isAllocationPossible() ==> false



✗ Allocation not possible with 111 pages – last book remains unassigned.

Move low to mid +1 i.e. 112

Step: 7

low = 112 high = 112 mid = 112

isAllocationPossible() ==> false



✗ Allocation not possible with 112 pages – last book remains unassigned.

Move low to mid +1 i.e. 113

Since low > high, binary search stops.



Max Pages = 113

Final Answer ==> 113 pages